



Byte Equality: Deterministic Work as a Market Primitive

Zach Kelling

Hanzo AI

2026

Abstract

The hardest problem in a decentralized compute market is not matching supply to demand. It is establishing that the supplier computed what the buyer paid for. Most designs answer this with replication or with succinct proofs, and both are expensive: replication multiplies the compute bill by the committee size, and proving multiplies it by three to six orders of magnitude.

For a large and underexploited class of work, neither is necessary. When a computation is bit-reproducible, correctness is decidable by comparison against a canonical implementation at the cost of running the canonical implementation once — and for a market that publishes fixed test vectors, at the cost of a table lookup. We describe the byte-equality contract we enforce across a 29-primitive cryptographic corpus and three GPU backends, argue that it is a market primitive rather than an engineering practice, and show that the crossover threshold it produces as a by-product is itself a quantity the market needs. We then draw the determinism boundary honestly: which AI work can be made bit-exact, at what throughput cost, and where it cannot.

Contents

1	Two regimes of verification	3
2	The contract	3
3	Why this is a market primitive	3
3.1	Fraud becomes impossible rather than detectable	4
3.2	Verification cost collapses	4
3.3	The crossover threshold is a price signal	4
4	The determinism boundary in AI work	4
4.1	What makes inference nondeterministic	4
4.2	What makes it deterministic again	5
4.3	Where it does not reach	5
5	Proposal: determinism class as a first-class job attribute	5
6	What is available	5

1 Two regimes of verification

Let a job be a function f and an input x ; the provider returns $\hat{y}$ and claims $\hat{y} = f(x)$. Verification splits on a single property.

Definition 1 (Determinism class). *A job is in class D if f is bit-reproducible: any correct implementation on any conforming device returns byte-identical output for identical input. It is in class S if f is well-defined only up to a tolerance — floating-point reductions whose order is unspecified, sampling with unpinned entropy, kernels whose tiling changes the summation order. It is in class O if the buyer cannot characterise f at all.*

The regimes have radically different verification economics, and a market that does not distinguish them overpays for the easy case and underprotects the hard one.

Class	Verification	Cost	Adjudication
D	recompute canonical / KAT lookup	$O(1)$ amortised	proof of misbehaviour
S	replication or succinct proof	$m \times$ or $10^3\text{--}10^6 \times$	statistical or cryptographic
O	reputation, attestation, stake	—	social

Table 1: Verification economics by determinism class.

The point of this paper is that class D is much larger than it looks, and that the machinery to exploit it already exists and is open source.

2 The contract

We hold every shipping GPU kernel to byte equality against a first-party CPU canonical. The contract has three parts, and each exists for a distinct reason.

Equality on randomized inputs. Every release, each primitive is exercised on $N = 1000$ randomized inputs across the CPU canonical and every GPU backend — Metal, CUDA and WGSL. All outputs must be byte-identical. Random inputs rather than fixed vectors matter because fixed vectors are what an implementation is tuned against; randomization is what catches the tiling bug that only fires on an odd tail.

Test-oracle isolation. The audited upstream implementation used as an oracle is never linked into the shipped library. This keeps the thing being tested and the thing testing it genuinely independent, so a shared bug cannot cancel out. Cross-oracle checking extends this: BN254 results are checked against **gnark-crypto** (36 of 36 vectors) and independently against **arkworks**, which are unrelated codebases and therefore give adversarial distance rather than a second opinion from a cousin.

Memory hygiene. Secret state does not leave attested accelerator memory on the hot path. This is orthogonal to correctness but belongs in the same contract because a market will run these kernels on machines the key owner does not control.

3 Why this is a market primitive

An engineering team reads the above as a quality-assurance regime. A market should read it as three distinct economic objects.

3.1 Fraud becomes impossible rather than detectable

Under replication, a dishonest provider is caught with probability depending on committee composition, and adjudication is a vote. Under byte equality a mismatch is a *proof*: the canonical output is a published fact, and any party can recompute the comparison. Slashing is then mechanical and requires no quorum, no oracle, and no assumption about the fraction of honest providers. This removes an entire class of governance from the protocol.

3.2 Verification cost collapses

Consider a market that would otherwise replicate three-fold for confidence. The verification overhead is 200% of the useful compute. Moving a job from class S to class D removes that overhead entirely. On a market where a nontrivial share of volume is cryptographic or integer-quantized work, this is the single largest efficiency available, and it is available without any new cryptography.

3.3 The crossover threshold is a price signal

A by-product of the same methodology is N^* , the batch size at which a GPU kernel overtakes the single-thread CPU baseline for a given primitive. Measured across the corpus, N^* ranges from 1 (homomorphic-encryption number theoretic transform — the GPU wins on a single call) to 1024 (SHA-256 — a GPU is worse than a CPU until a thousand hashes are in flight).

That number is not a benchmark curiosity. It is the feasibility boundary of a bid. A provider offering GPU capacity for a class-D job with batch size below N^* is offering something strictly worse than a commodity CPU at a higher price. Publishing N^* per primitive lets the market reject mispriced supply structurally rather than discovering it through disappointed buyers.

Primitive family	Measured speedup	Crossover N^*
Fused BLS aggregation	144.22×	—
FHE NTT (Metal, kernel-only, 10-iter median)	16.92×	1
Batched Ed25519 verify (RFC 8032) at $N=4096$	26.7×	—
SHA-256	—	1024

Table 2: Representative measurements from the corpus, single Apple M2 Ultra. CPU NTT throughput at $N=2^{20}$ is 6–9 Mops/s on the same device.

4 The determinism boundary in AI work

The natural objection is that neural inference is class S and therefore none of this applies. That is true by default and false by construction, and the distinction is worth money.

4.1 What makes inference nondeterministic

Floating-point addition is not associative. A reduction whose order depends on block scheduling, an atomic accumulation whose interleaving depends on occupancy, a flash-attention tiling chosen by an autotuner at load time — each makes output depend on hardware and scheduling rather than on f alone. Batch size changes reduction order, so even the same device gives different answers at different loads. None of this is error; it is unspecified behaviour that was never worth specifying.

4.2 What makes it deterministic again

Determinism is a kernel property one may choose to pay for:

- **Integer paths.** Weight-and-activation quantized inference in `int8` or `int4` with integer accumulation is exactly reproducible. There is no rounding order to disagree about.
- **Batch-invariant kernels.** Fixing the reduction order independent of batch size and occupancy removes the load dependence, at a throughput cost typically in the low tens of percent.
- **Pinned sampling.** Greedy decoding, or sampling from a specified PRF seeded by a value in the job specification, makes the token sequence a function of the request.
- **Pinned numerics.** A declared accumulation dtype and a declared attention implementation, both carried in the job specification and attested by the provider.

The result is a job class we would write as D_{inf} : slower than unconstrained inference, and verifiable by recomputation on any conforming device. Whether a buyer wants it is a price question, which is the right way for it to be.

4.3 Where it does not reach

Training is not practically reproducible across heterogeneous hardware, and large-batch `bf16` inference at maximum throughput is not either. Those remain class `S` and need the tier ladder. The claim here is narrower and defensible: a material fraction of paid compute — all cryptographic work, all quantized inference, all embedding generation with pinned numerics, all search — can be moved into class `D` deliberately.

5 Proposal: determinism class as a first-class job attribute

We suggest a market carry the class on the job rather than inferring it.

```
job:
  kind: inference | crypto | search
  determinism: D | D_inf | S | 0
  numerics: { dtype: int8, accum: int32, attention: fixed, sampling: greedy }
  canonical: <content-addressed reference implementation>
  vectors: <published KAT set, optional>
```

Three consequences follow immediately. Pricing differs by class because verification cost differs by class, and the buyer chooses. Providers advertise which classes they can honour, which is a capability constraint the allocator already understands. And a class-`D` job needs no proof system at all, which means a market can launch with real verifiable volume before any proving infrastructure exists.

6 What is available

The corpus, the contract and the measurement harness are open source under permissive licence: a 29-primitive library spanning hashes, AEAD, elliptic curves including pairing-friendly ones, polynomial commitments, lattice primitives, post-quantum signatures and key encapsulation, threshold schemes, homomorphic-encryption building blocks and EVM precompile wrappers, with Metal, CUDA and WGS kernel held to the equality contract described above.

Adopting it does not require adopting anything else in our stack. The contract is a testing discipline plus a published artifact; the artifact is what a market needs, and it is reproducible by anyone who wants to check it.

Note on scope

Nothing here establishes that a provider ran the job *when* it said, on the hardware it claimed, or without also serving someone else. Byte equality establishes what was computed, not the circumstances. Those are separate questions with separate mechanisms, treated in the companion papers on verification tiers and on settlement.